



nextwork.org

Deploy Backend with Kubernetes



yakubu abdullahi yusuf

```
ec2-user@ip-172-31-36-76 ~]$ aws eks update-kubeconfig --name nextwork-eks-cluster --region us-east-2
ec2-user@ip-172-31-36-76 ~]$ kubectl apply -f flask-deployment.yaml
ec2-user@ip-172-31-36-76 ~]$ kubectl apply -f flask-service.yaml
ec2-user@ip-172-31-36-76 ~]$ █
```



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will set up the backend of my application for deployment, I will install kubectl, and also deploy the backend on a Kubernetes cluster.

Tools and concepts

I used Amazon EKS, Git, ECS, Docker, GitHub, EC2 instances, kubectl, and CloudFormation to push and manage the backend application in a cloud environment. These tools allowed me to version control the code, containerize the application, and store and retrieve Docker images efficiently. CloudFormation helped me provision the required AWS infrastructure in a consistent way, while kubectl enabled me to deploy and manage Kubernetes resources such as pods, deployments, and services within the EKS cluster.

Project reflection

This project took me approximately forty minutes.



Backend Deployment!

To deploy my backend application, I used kubectl to apply the Deployment and Service YAML files, which created the backend pods and exposed the application through a Kubernetes Service.

kubectl

kubectl is a command-line tool used to interact with and manage Kubernetes clusters. I need this tool to deploy applications and manage resources such as pods and services in the cluster. I can't use eksctl for this job because eksctl is mainly used for creating and managing EKS clusters, not for deploying and controlling workloads inside the cluster.

```
ec2-user@ip-172-31-36-76 ~]$ aws eks update-kubeconfig --name nextwork-eks-cluster --region us-east-2
ec2-user@ip-172-31-36-76 ~]$ kubectl apply -f flask-deployment.yaml
ec2-user@ip-172-31-36-76 ~]$ kubectl apply -f flask-service.yaml
ec2-user@ip-172-31-36-76 ~]$ █
```



Verifying Deployment

My extension for this project is to use the EKS console to monitor the cluster. I had to set up IAM access policies because EKS requires permission-based authentication, and without the right IAM role/user permissions I wouldn't be able to connect to the cluster or manage resources with kubectl. I set up access by attaching the required EKS permissions to my IAM user/role through the EC2 terminal.

Once I gained access to my cluster's nodes, I discovered pods running inside each node. Pods are the smallest deployable units in Kubernetes and are used to run one or more containers together. Containers in a pod share the same network and storage resources, which allows them to communicate efficiently and operate as a single application unit.

The EKS console shows the events for each pod, where I could see details about pod creation, image pulling, and container startup. This validated that the application was deployed successfully and that Kubernetes was managing the pods as expected.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Events (5)

Type	Reason	Event time	From	Message	Inspect with Amazon Q
✔ Normal	Scheduled	an hour ago	default-scheduler	Successfully assigned default/nextwork-flask-backend-544d767557-tz7kq to ip-192-168-24-170.us-east-2.compute.internal	-
✔ Normal	Pulling	an hour ago	kubelet	Pulling image "176971016201.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend-ecr:latest"	-
✔ Normal	Pulled	an hour ago	kubelet	Successfully pulled image "176971016201.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend-ecr:latest" in 2.52s (2.52s including waiting). Image size: 39996408 bytes.	-
✔ Normal	Created	an hour ago	kubelet	Created container: nextwork-flask-backend	-
✔ Normal	Started	an hour ago	kubelet	Started container nextwork-flask-backend	-



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

